

A Compiler Framework for Packing Optimization & TPU Acceleration in Fully Homomorphic Encryption

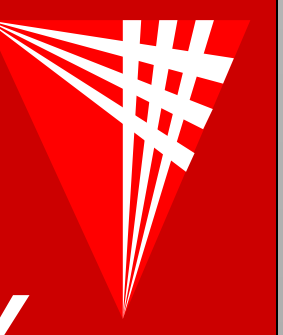
Zohaib Azam^{*1}, Akash Madhu^{*1}, Jianming Tong², Minxuan Zhou¹.

¹ Illinois Institute of Technology, Department of Computer Science,

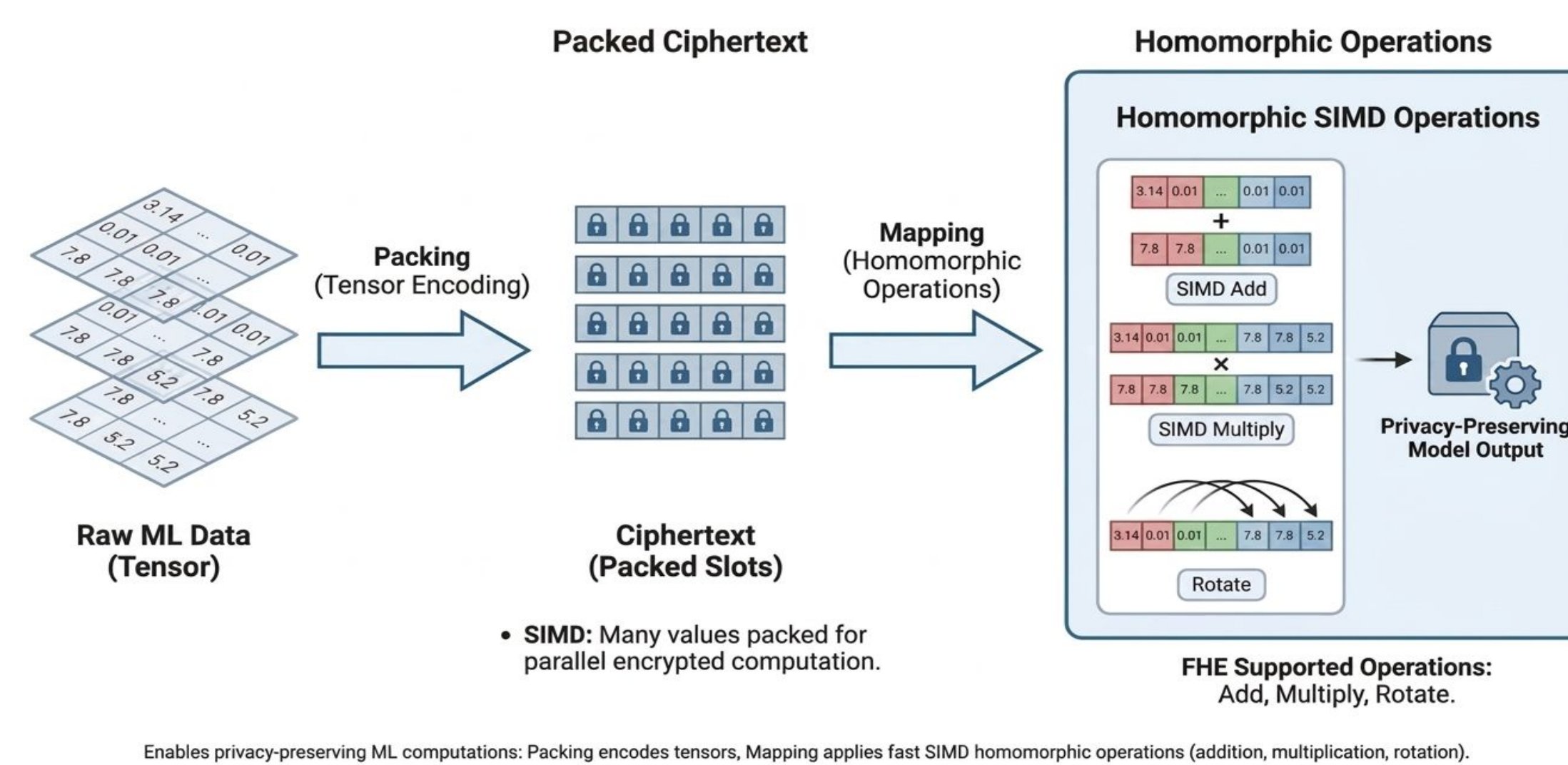
² Georgia Institute of Technology, Department of Computer Science



ILLINOIS INSTITUTE
OF TECHNOLOGY



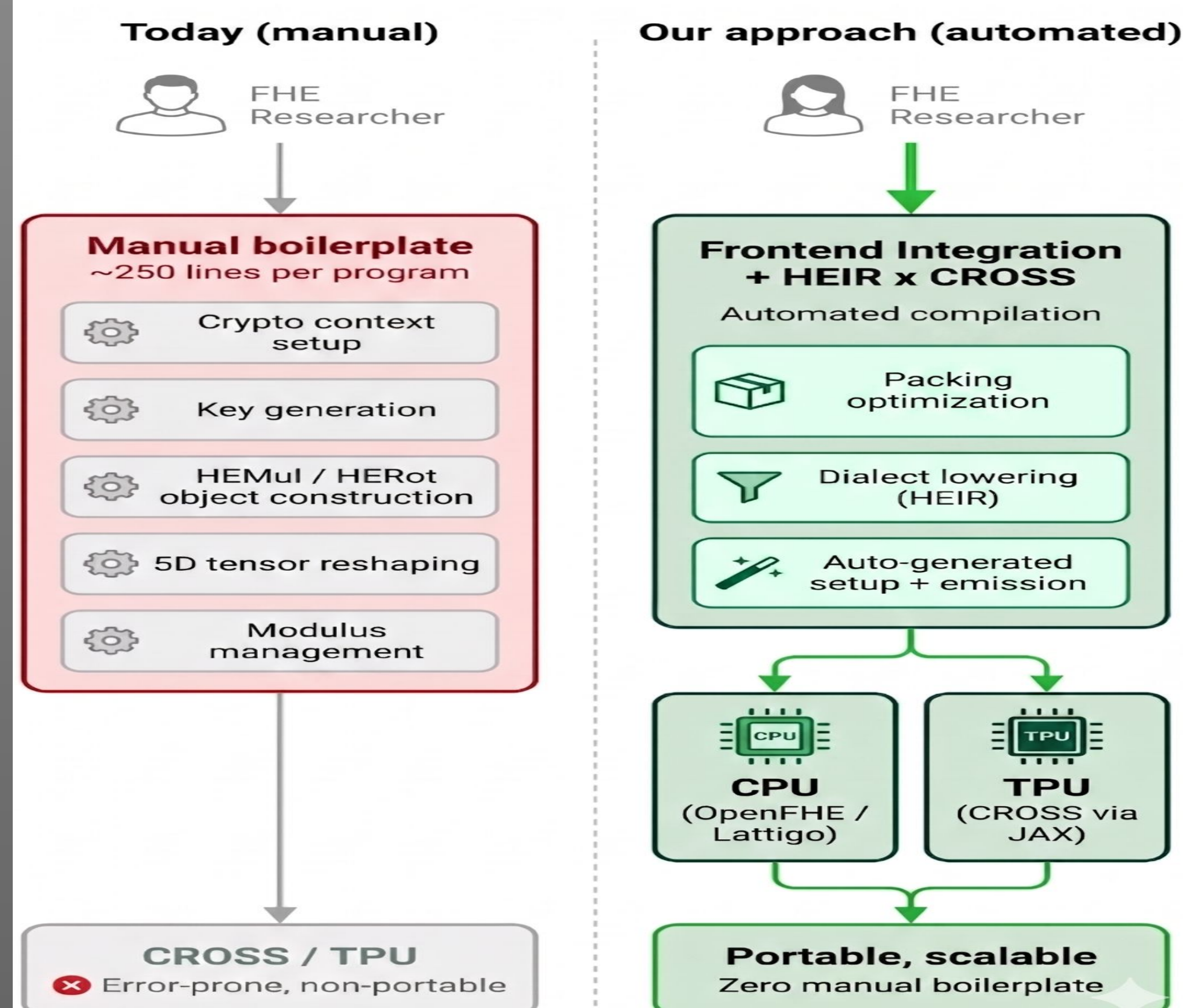
Abstract



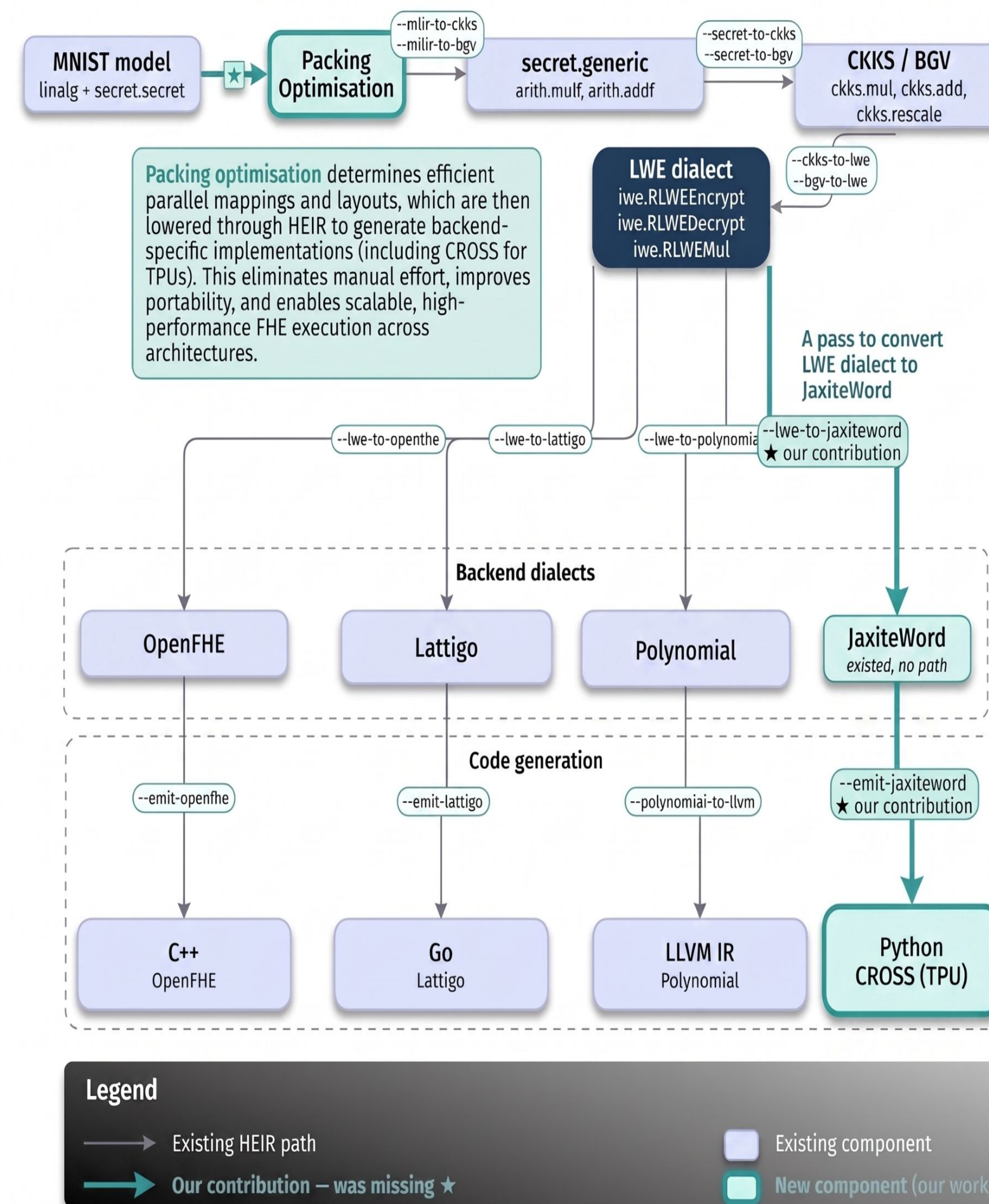
Our framework integrated into HEIR addresses both gaps.

- Enables layout optimization
- Adds a new backend targeting CROSS, a CKKS library for TPUs

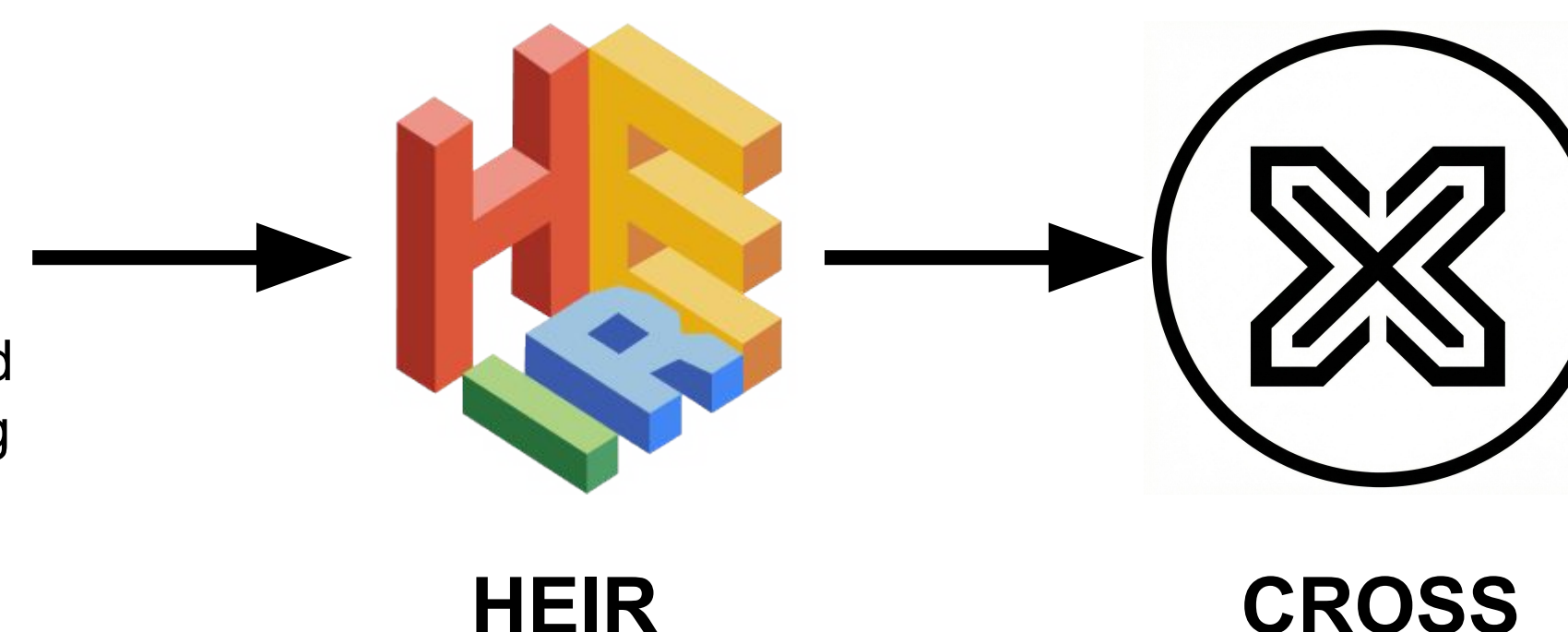
Problem & Solution



Approach & Integration



Layout define index-to-slot mappings (e.g., [b, ci, hi, wi] → [ct, slot]) for tensor packing, while the computation schedule is encapsulated in a secret wrapper using arith and tensor dialects.



IRs & their Benefits

Stage 1 - Packing & Mapping

```
#input_layout = #tensor_ext.layout<"[b, ci, hi, wi] -> [ct, slot] : ... ">
#output_layout = #tensor_ext.layout<"[b, co, ho, wo] -> [ct, slot] : ... ">
```

The layout decision determines how tensors are packed.

```
#input_original_type =
#tensor_ext.original_type<
originalType = tensor<1x1x4x4xf32>,
layout = #input_layout
>#output_original_type =
#tensor_ext.original_type<
originalType = tensor<1x2x2x2xf32>,
layout = #output_layout>
```

Layout-aware types drive the computation schedule.

```
func.func @simple_conv {
  // computation schedule based on the optimised layout
}
```

Stage 2-Mid IR (CKKS)

```
func.func @simple_conv(
  %arg0: tensor<1x!\ct\_L1>,
) -> tensor<1x!\ct\_L0> {
  %c0 = arith.constant 0 : index
  %a = tensor.extract %arg0[%c0]
  %b = tensor.extract %arg1[%c0]
```

```
// Multiply: degree-3 ct
%ct = ckks.mul %a, %b: (!ct\_L1, !ct\_L1) -> !ct\_L1\_D3
// Relinearize: degree 2
%ct\_1 = ckks.relinearize %ct
{from\_basis = array<i32: 0,1,2>, to\_basis = array<i32: 0,1>}
:(!ct\_L1\_D3) -> !ct\_L1
// Rescale: L1→L0
%ct\_2 = ckks.rescale %ct\_1: !ct\_L1 -> !ct\_L0
%inserted = tensor.insert %ct\_2 ...
return %inserted : tensor<1x!\ct\_L0>
```

- Explicit Noise Management
- Optimizing Opportunities

Last Stage - CROSS Code

```
def simple_conv(ctx, eval_key, arg0, arg1):
  v0, v1 = arg0[0], arg1[0]
  v2 =
  ctx.he_mul[1].mul_no_relin(v0, v1)
  v3 = ctx.he_mul[1].relin(v2, eval_key)
  v4 = ctx.he_mul[1].mod_reduce(v3)
  return [v4]
```

- Just computation in inference method
- Receives TPU aware parameters via context methods configured offline
- Runnable on TPU

Conclusion & Future Work

- We validate HEIR-emitted CROSS against manually implemented CROSS code and confirm correctness on a homomorphic multiplication microbenchmark. **HEIR-emitted CROSS** achieves **118.8 ms** latency versus **142.8 ms** for **direct CROSS**, giving a 16.8% latency reduction.
- Future work will extend this workflow to end-to-end neural networks and additional targets, including GPU backends.